

webdriver — automatisation WebDriver en Lua avec Babet

babet-webdriver est une **bibliothèque Lua** d'automatisation de navigateur basée sur le standard W3C WebDriver. Les scripts d'automatisation sont écrits directement en **Lua** et exécutés par Babet. `webdriver.lua` fournit **WebDriver Classic** avec Babet 2.9.0 ou supérieur ; depuis la version 2.0.0, `webdriver_bidi.lua` ajoute **WebDriver BiDi** grâce au client WebSocket natif de Babet 2.22.0 ou supérieur. La bibliothèque pilote Firefox, Chrome et Chromium à travers `geckodriver` ou `chromedriver`. Babet est le runtime qui exécute les scripts Lua et fournit les API natives HTTP, WebSocket, processus et workers utilisées par `babet-webdriver`. Il est disponible sur son [dépôt GitHub officiel](#).

Sommaire

- [Installation](#)
- [Démarrer un nouveau projet](#)
- [Créer une session](#)
- [Options de démarrage](#)
- [Cycle de vie du driver](#)
- [Navigation](#)
- [Recherche et attentes](#)
- [Éléments](#)
- [JavaScript](#)
- [Actions clavier, souris et molette](#)
- [Fenêtres, onglets et frames](#)
- [Alertes, cookies et timeouts](#)
- [Captures d'écran](#)
- [Gestion automatique des drivers](#)
- [WebDriver BiDi](#)
- [Workers et channels](#)
- [Contrat d'erreur](#)
- [Limites](#)

Installation

Les fichiers `webdriver.lua`, `webdriver_version.lua`, `driver_manager.lua` et, selon le mode utilisé, `webdriver_worker.lua`, `webdriver_bidi.lua` et `webdriver_bidi_worker.lua` doivent être accessibles par `package.path`.

```
local webdriver = require("webdriver")
```

La bibliothèque requiert :

- [Babet 2.9.0 ou supérieur](#) pour WebDriver Classic ;
- Babet 2.22.0 ou supérieur pour WebDriver BiDi et la suite de tests complète 2.0 ;
- un système Linux ;
- Firefox, Chrome ou Chromium ;
- `geckodriver` ou `chromedriver`, installé ou téléchargeable.

Elle n'a pas besoin de `tar`, `unzip`, `curl`, `base64` ou d'un shell externe.

Démarrer un nouveau projet

Quoi télécharger

Un nouveau projet a besoin de deux éléments indépendants :

1. **babet-webdriver** : télécharge l'archive source de la version voulue depuis [les releases babet-webdriver](#). Le runtime est constitué de modules Lua : il n'y a pas d'installation séparée.
2. **Babet** : télécharge un binaire Linux depuis [les releases Babet](#), ou compile-le depuis le [dépôt Babet](#). Utilise Babet **2.22.0 ou supérieur** pour disposer de toutes les fonctions de `babet-webdriver` 2.0. Un projet utilisant uniquement WebDriver Classic peut fonctionner

à partir de **Babet 2.9.0**. Sur un PC Linux Intel/AMD 64 bits classique, choisis l'artefact `babet-<version>-linux-x86_64` ; les builds ARM sont publiés séparément.

Si tu ne veux pas choisir les modules un par un, le plus simple et le plus sûr est de copier tous les fichiers runtime :

```
mon-projet/
├── bin/
│   └── babet
├── webdriver.lua
├── webdriver_version.lua
├── driver_manager.lua
├── webdriver_worker.lua
├── webdriver_bidi.lua
├── webdriver_bidi_worker.lua
└── main.lua
```

```
chmod +x bin/babet
./bin/babet main.lua
```

L'emplacement `bin/babet` est facultatif : Babet peut aussi être installé dans le `PATH` ou appelé avec un chemin absolu quelconque. De même, les modules Lua peuvent être rangés dans un autre dossier à condition d'ajouter celui-ci à `package.path`. Les éléments de développement (`tests/`, `examples/`, `docs/`, `tools/`, `run_*.sh`, `build_docs.sh`) ne sont pas nécessaires dans un projet utilisateur.

`geckodriver` et `chromedriver` ne font partie d'aucune des deux archives source. Ils peuvent être présents dans le `PATH`, indiqués avec `driver_path`, ou installés par `driver_manager.lua`. Un premier téléchargement encore inconnu est refusé sauf s'il est explicitement autorisé avec `trust_on_first_use = true`, ou contrôlé avec `expected_sha256`.

Projet direct : fichiers indispensables

Pour piloter un navigateur depuis le script principal, copie uniquement :

```
mon-projet/
├── webdriver.lua
├── webdriver_version.lua
├── driver_manager.lua
└── main.lua
```

`webdriver.lua` charge `webdriver_version.lua` et `driver_manager.lua` en interne. Les trois modules sont donc requis même si `geckodriver` ou `chromedriver` est déjà installé.

Avec les modules à côté de `main.lua`, le chargement reste direct :

```
local webdriver = require("webdriver")
```

Projet avec worker

Pour héberger la session WebDriver dans un worker et communiquer par channels, ajoute le proxy :

```
mon-projet/
├── webdriver.lua
├── webdriver_version.lua
├── driver_manager.lua
├── webdriver_worker.lua
└── main.lua
```

Le script principal chargera alors :

```
local webdriver_worker = require("webdriver_worker")
```

Pour BiDi direct, ajoute `webdriver_bidi.lua`. Pour un transport BiDi dans un worker dédié, ajoute aussi `webdriver_bidi_worker.lua`. Une session Classic dans `webdriver_worker.lua` utilise ce même module lorsqu'on appelle `session:bidi()`.

Le fichier `tools/check_env.lua` est un outil de diagnostic facultatif. Les dossiers `tests/`, `examples/`, `docs/`, ainsi que `build_docs.sh` et les scripts `run_*.sh`, ne sont pas nécessaires dans un projet utilisateur.

Emplacement du binaire Babet

Le dossier `bin/` utilisé par le dépôt n'est qu'une convention pratique. Babet peut être placé n'importe où. Il peut être téléchargé ou compilé depuis son [dépôt GitHub](#), et la version minimale requise par la partie Classic est la **2.9.0**. WebDriver BiDi nécessite **Babet 2.22.0**.

Binaire local au projet :

```
mon-projet/  
├── bin/babet  
├── webdriver.lua  
├── webdriver_version.lua  
├── driver_manager.lua  
└── main.lua
```

```
./bin/babet main.lua
```

Babet installé dans le PATH :

```
babet main.lua
```

Avec un shebang, cela permet aussi :

```
#!/usr/bin/env babet
```

```
chmod +x main.lua  
./main.lua
```

Babet situé à un chemin quelconque :

```
/opt/babet/bin/babet main.lua
```

Dans le dépôt `babet-webdriver`, les scripts `run_tests.sh`, `run_smoke.sh`, `run_worker_smoke.sh` et `run_all_tests.sh` cherchent le binaire dans cet ordre :

1. chemin fourni par BABET ;
2. `bin/babet` ;
3. commande `babet` disponible dans le PATH.

Exemple :

```
BABET=/opt/babet/bin/babet ./run_tests.sh
```

Pour lancer toute la campagne de validation en une seule commande :

```
./run_all_tests.sh
```

Le script active le mode headless par défaut, affiche le déroulé en direct dans le terminal et copie exactement la même sortie dans `babet-webdriver-tests.txt`. Le journal est d'abord écrit dans un fichier temporaire unique du même répertoire, puis publié atomiquement à la fin de la campagne. Un journal complet existant n'est donc jamais tronqué pendant les tests et deux campagnes accidentellement parallèles ne mélangent pas leurs contenus. Si plusieurs campagnes visent le même fichier, celle qui termine en dernier devient simplement le journal courant. Une campagne terminée, réussie ou en échec, remplace ainsi le journal précédent afin de pouvoir être transmise telle quelle. `HEADLESS=0` permet d'afficher les navigateurs et `TEST_LOG=/chemin/test.txt` permet de choisir un autre fichier.

Exemple minimal complet

```
#!/usr/bin/env babet

local webdriver = require("webdriver")

local driver, err = webdriver.firefox({ headless = true })
if not driver then
    io.stderr:write(tostring(err), "\n")
    os.exit(1)
end

local ok, run_err = xpcall(function()
    assert(driver:open("https://example.com"))
    print(assert(driver:title()))

    local heading = assert(driver:css("h1"))
    print(assert(heading:text()))
end, debug.traceback)

local quit_ok, quit_err = driver:quit()
if not quit_ok then
    io.stderr:write("Fermeture WebDriver : ", tostring(quit_err), "\n")
end

if not ok then
    io.stderr:write(tostring(run_err), "\n")
    os.exit(1)
end
```

Modules dans un sous-dossier

Si tu préfères ranger les modules dans lib/ :

```
mon-projet/
├── lib/
│   ├── webdriver.lua
│   ├── webdriver_version.lua
│   ├── driver_manager.lua
│   └── webdriver_worker.lua
└── main.lua
```

Ajoute ce chemin avant le premier require :

```
package.path = "./lib/?.lua;" .. package.path
```

```
local webdriver = require("webdriver")
```

Créer une session

Firefox

```
local driver, err = webdriver.firefox({
    headless = true,
})
assert(driver, err)
```

Chrome ou Chromium

```
local chrome = assert(webdriver.chrome({ headless = true }))
local chromium = assert(webdriver.chromium({ headless = true }))
```

Les helpers ne modifient jamais la table d'options fournie par l'appelant.

Forme générique

```
local driver = assert(webdriver.start({
    browser = "firefox",
    headless = true,
}))
```

Options de démarrage

Navigateur et capacités

Option	Type	Défaut	Description
browser	chaîne	firefox	firefox, chrome, chromium
headless	booléen	false	ajoute l'argument headless adapté
args	tableau dense	{}	arguments du navigateur
binary	chaîne	auto pour Chromium	binaire spécifique du navigateur
user_data_dir	chaîne	absent	profil explicite Chrome/Chromium
window_size	{w,h}	absent	taille de fenêtre initiale
accept_insecure_certs	booléen	false	capability W3C correspondante
bidirectional	booléen	false	demande webSocketUrl et active la négoc

```
local driver = assert(webdriver.firefox({
    headless = true,
    args = { "--private-window" },
    binary = "/usr/lib/firefox/firefox",
    window_size = { 1600, 1000 },
    accept_insecure_certs = false,
}))
```

Pour `webdriver.chromium()`, le binaire est recherché dans PATH sous les noms `chromium` puis `chromium-browser`. Pour `webdriver.chrome()`, la bibliothèque essaie `google-chrome-stable`, `google-chrome` puis `chrome`, mais laisse `ChromeDriver` appliquer sa détection normale si aucun de ces noms n'est trouvé.

Chromium distribué en Snap

`ChromeDriver` crée normalement un profil temporaire. Avec le Chromium Snap, ce profil peut se retrouver hors de l'espace visible par le confinement du snap, ce qui provoque notamment `DevToolsActivePort file doesn't exist`.

Lorsque le binaire détecté se trouve sous `/snap/bin/` ou `/snap/chromium/`, la bibliothèque crée donc automatiquement un profil temporaire sous :

```
~/snap/chromium/common/babet-webdriver/profiles/
```

Ce profil est transmis avec `--user-data-dir=...` puis supprimé lors de `driver:quit()`. Aucun `--no-sandbox` n'est ajouté : la sandbox reste active.

Un profil explicite reste possible :

```
local driver = assert(webdriver.chromium({
    headless = true,
```

```
user_data_dir = os.getenv("HOME") .. "/snap/chromium/common/mon-profil",
}))
```

user_data_dir ne peut pas être combiné avec un argument --user-data-dir=... placé manuellement dans args. Un profil fourni par l'appelant n'est jamais supprimé automatiquement.

Driver externe

Option	Type	Défaut	Description
driver_path	chaîne	recherche PATH	chemin du driver
auto_install	booléen	true	installe si absent
trust_on_first_use	booléen	false	autorise le premier artefact non pinné
expected_sha256	chaîne hex	absent	hash attendu de l'archive
platform	chaîne	auto	plateforme forcée
cache	chaîne	cache par défaut	dossier du cache
force_driver_download	booléen	false	force une réinstallation

Port et connexion

Option	Type	Défaut	Description
port	entier	automatique	port du serveur WebDriver
port_attempts	entier	5	essais pour un port automatique
attach	booléen	false	utilise un driver externe prêt à créer une nouvelle session
start_timeout	secondes	15	budget de démarrage
status_timeout	secondes	1	timeout de chaque sonde /status
poll_interval	secondes	0.1	intervalle entre sondes

Sans port explicite, la bibliothèque demande un port libre au noyau. La socket temporaire doit ensuite être fermée avant le lancement du driver ; une petite fenêtre de course subsiste donc. port_attempts permet de recommencer si un autre processus prend le port entre-temps.

En mode attach, le driver externe doit répondre à /status avec ready=true : il doit donc être disponible pour créer **une nouvelle session**. Ce mode ne reprend pas une session Selenium existante. En particulier, geckodriver renvoie normalement ready=false lorsqu'il est déjà occupé ; babet-webdriver le signale alors comme joignable mais indisponible, plutôt que comme un problème réseau.

En mode attach, les défauts historiques sont conservés :

- Firefox : 4444 ;
- Chrome/Chromium : 9515.

```
local driver = assert(webdriver.firefox({
  attach = true,
  port = 4444,
}))
```

HTTP, captures et logs

Option	Défaut	Description
request_timeout	120 s	timeout des commandes WebDriver
max_body_size	64 Mio	limite d'une réponse HTTP
screenshot_max_size	64 Mio	limite après décodage Base64
screenshot_permissions	0644	permissions du PNG publié
screenshot_durable	true	fsync du fichier et du dossier
print_max_size	64 Mio	limite du PDF après décodage Base64

Option	Défaut	Description
print_permissions	0644	permissions du PDF publié
print_durable	true	fsync du PDF et du dossier
log_path	automatique	fichier de log exact
log_dir	cache/logs	dossier des logs automatiques
log_append	true	ajoute au log au lieu de tronquer
log_permissions	0600	permissions à la création
terminate_grace	2 s	grâce avant SIGKILL

Les options sont strictes. `timeout = 5`, par exemple, lève immédiatement une erreur Lua.

Cycle de vie du driver

Quand la bibliothèque lance le driver, elle conserve l'objet retourné par `babet.spawn()` :

```
print(driver:pid())
print(driver:port())
print(driver:log_path())
print(driver:is_running())
```

Les flux sont configurés ainsi :

- stdin vers `/dev/null` ;
- stdout vers le fichier de log ;
- stderr fusionné dans stdout.

Aucun shell n'interprète le chemin du binaire ni ses arguments.

Fermeture

```
local ok, err = driver:quit()
assert(ok, err)
```

`quit()` :

1. demande la suppression de la session W3C ;
2. envoie SIGTERM au groupe du processus driver ;
3. attend `terminate_grace` ;
4. emploie SIGKILL si nécessaire ;
5. ferme les descripteurs Babet.

La méthode est idempotente. `driver:close()` est un alias. L'objet possède aussi un métaméthode `__close` pour les variables Lua to-be-closed.

Navigation

```
assert(driver:open("https://example.com"))
print(assert(driver:url()))
print(assert(driver:title()))
print(assert(driver:source()))
assert(driver:back())
assert(driver:forward())
assert(driver:refresh())
```

Recherche et attentes

Stratégies

```

driver:find("main h1") -- CSS
driver:find("//main//h1", { by = "xpath" })
driver:find("submit", { by = "id" })
driver:find("email", { by = "name" })
driver:find("button", { by = "tag" })
driver:find("active", { by = "class" })
driver:find("Accueil", { by = "link" })
driver:find("Acc", { by = "plink" })

```

Les raccourcis sont :

```

driver:css(selector)
driver:xpath(selector)
driver:id(value)
driver:name(value)
driver:tag(value)

```

id, name et class sont transformés en sélecteurs CSS correctement échappés.

Un élément

```

local element, err = driver:css("h1")
assert(element, err)

```

En cas d'absence, le serveur renvoie l'erreur W3C no such element.

Plusieurs éléments

```

local elements = assert(driver:find_all("article"))
for _, element in ipairs(elements) do
    print(assert(element:text()))
end

```

Une liste vide est un succès normal.

Tester la présence

```

local exists, err = driver:exists("#optional")
assert(exists ~= nil, err)
if exists then
    print("présent")
end

```

Seule l'erreur no such element devient false. Une panne réseau, une session fermée ou un driver mort reste une erreur.

Attendre un état

```

local button = assert(driver:wait("#submit", {
    state = "clickable",
    timeout = 20,
    interval = 0.2,
}))

```

États :

- present : élément trouvé ;
- visible : trouvé et affiché ;
- clickable : affiché et activé ;

- gone : absence confirmée.

Par défaut, une référence devenue stale provoque une nouvelle recherche. Les autres erreurs sont propagées immédiatement.

Attente libre

```
local result = assert(driver:wait_until(function()
    local value, err = driver:js("return window.appReady === true")
    if value == nil and err then return nil, err end
    return value
end, 30, 0.25))
```

Une exception du callback ou une erreur renvoyée en seconde valeur n'est pas silencieusement ignorée.

Éléments

Actions simples

```
assert(element:click())
assert(element:clear())
assert(element:type("Bonjour"))
assert(element:submit())
```

submit() recherche le formulaire englobant en JavaScript et préfère requestSubmit() lorsqu'il est disponible.

Lecture

```
print(assert(element:text()))
print(assert(element:tag()))
print(assert(element:rect()).width)
print(assert(element:css("display")))
print(assert(element:property("value")))
print(assert(element:dom_attr("data-id")))
print(assert(element:attr("textContent")))
print(assert(element:value()))
print(assert(element:computed_role()))
print(assert(element:computed_label()))
```

attr() imite le comportement pratique de Selenium : il tente l'attribut HTML, puis la propriété DOM. Si ni l'un ni l'autre n'existe, il renvoie nil sans erreur. Les résultats JavaScript génériques continuent, eux, de préserver babet.json.null.

États booléens

```
local displayed, err = element:displayed()
assert(displayed ~= nil, err)
```

displayed, enabled et selected propagent les erreurs. Elles ne changent plus une erreur en false.

Recherche imbriquée

```
local row = assert(driver:css("tr.active"))
local button = assert(row:find("button.save"))
local cells = assert(row:find_all("td"))
```

Shadow DOM W3C :

```
local host = assert(driver:css("my-component"))
local shadow = assert(host:shadow_root())
assert(webdriver.is_shadow_root(shadow))
local button = assert(shadow:find("button.primary"))
local items = assert(shadow:find_all("li"))
```

L'élément actuellement actif est accessible avec `driver:active_element()`.

JavaScript

```
local text = assert(driver:js(
    "return arguments[0].textContent",
    element
))
```

Les objets `Element` présents dans les arguments sont sérialisés avec la clé `W3C`. Les éléments renvoyés par le script sont reconstruits automatiquement, y compris au sein d'une table imbriquée.

Les tables cycliques sont refusées avant l'envoi. Une valeur JavaScript `null` est conservée sous la forme `babet.json.null`, ce qui la distingue d'un `nil` d'erreur. La variante asynchrone W3C est exposée par `js_async()` :

```
local value = assert(driver:js_async([[
    const done = arguments[arguments.length - 1];
    setTimeout(() => done("prêt"), 10);
]]))
```

Actions clavier, souris et molette

```
assert(driver:actions()
    :move_to(element)
    :click()
    :send_keys("Bonjour")
    :perform())
```

Méthodes :

```
move_to(element [, x, y])
move_by(x, y)
click([element])
double_click([element])
context_click([element])
click_and_hold([element])
release()
key_down(character)
key_up(character)
send_keys(text)
pause(milliseconds)
scroll(delta_x, delta_y [, opts])
drag_and_drop(source, destination)
perform()
clear()
```

Le builder aligne les sources clavier, souris et molette tick par tick. `scroll()` émet une source W3C `wheel` ; son origine peut être `"viewport"` ou un `Element`, avec `x`, `y` et `duration` facultatifs. Les coordonnées, deltas et la durée sont des entiers conformément au protocole ; `move_to()`, `move_by()` et `pause()` appliquent la même validation stricte. `perform()` remet les séquences à zéro.

Les touches spéciales sont exposées dans `webdriver.keys`, par exemple :

```
webdriver.keys.ENTER
webdriver.keys.CONTROL
webdriver.keys.DELETE
webdriver.keys.F1
webdriver.keys.SEPARATOR
webdriver.keys.RIGHT_SHIFT
webdriver.keys.RIGHT_CONTROL
webdriver.keys.RIGHT_ALT
webdriver.keys.RIGHT_META
```

Fenêtres, onglets et frames

```
local current = assert(driver:window())
local handles = assert(driver:windows())
assert(driver:switch(handles[#handles]))
assert(driver:switch_last())
local new_handle = assert(driver:new_tab())
local window_handle, kind = assert(driver:new_window("window"))
assert(kind == "window")
assert(driver:close_window())
```

Rectangles :

```
assert(driver:set_window_rect({ x = 0, y = 0, width = 1280, height = 900 }))
local rect = assert(driver:window_rect())
assert(driver:maximize())
assert(driver:minimize())
assert(driver:fullscreen())
```

Frames :

```
assert(driver:frame(0))
assert(driver:frame(frame_element))
assert(driver:parent_frame())
assert(driver:top_frame())
```

Alertes, cookies et timeouts

Alertes

```
print(assert(driver:alert_text()))
assert(driver:alert_send("réponse"))
assert(driver:accept_alert())
assert(driver:dismiss_alert())
```

Cookies

```
local cookies = assert(driver:cookies())
local theme = assert(driver:cookie("theme"))
assert(driver:set_cookie({ name = "theme", value = "dark" }))
assert(driver:delete_cookie("theme"))
assert(driver:clear_cookies())
```

Le nom d'un cookie est encodé comme segment d'URL ; les /, espaces et caractères non réservés ne peuvent pas casser le chemin WebDriver.

Timeouts W3C

Les valeurs sont exprimées en secondes :

```
assert(driver:set_timeouts({
    implicit = 0,
    page_load = 60,
    script = 30,
}))
local timeouts = assert(driver:get_timeouts())
print(timeouts.page_load)

-- W3C autorise null pour supprimer une limite lorsque le driver le supporte.
assert(driver:set_timeouts({ script = babet.json.null })))
```

Captures d'écran

Sans chemin, la chaîne Base64 est renvoyée :

```
local encoded = assert(driver:screenshot())
local element_encoded = assert(element:screenshot())
```

Avec un chemin, Babet décode les données binaires et publie le fichier atomiquement :

```
assert(driver:screenshot("captures/page.png"))
assert(element:screenshot("captures/button.png"))
```

Le dossier parent est créé récursivement si nécessaire. Grâce à la publication atomique, un lecteur voit l'ancien fichier complet ou le nouveau, jamais un PNG partiel.

L'impression W3C renvoie un PDF en Base64 ou le publie atomiquement lorsqu'un chemin est fourni :

```
local encoded_pdf = assert(driver:print({ orientation = "portrait" }))
assert(driver:print({
    orientation = "landscape",
    background = true,
    page = { width = 21, height = 29.7 },
    margin = { top = 1, bottom = 1, left = 1, right = 1 },
    page_ranges = { "1-2", 4 },
}, "captures/page.pdf"))
```

Gestion automatique des drivers

driver_manager.lua peut être utilisé directement :

```
local manager = require("driver_manager")
local path = assert(manager.install("firefox", {
    trust_on_first_use = true,
})))
```

Résolution

- geckodriver : dernière release GitHub officielle, sauf driver_manager.gecko_version forcée ;
- chromedriver : version du milestone du binaire Chrome ou Chromium réellement sélectionné, sauf driver_manager.chrome_version forcée.

Vérification

Le cache utilise XDG_CACHE_HOME/babet-webdriver lorsqu'il est défini, sinon :

```
~/.cache/babet-webdriver/pins/<driver>_<plateforme>_<version>.json
```

Sans HOME, un cache privé par utilisateur/UID est choisi sous /tmp afin d'éviter les collisions de permissions entre comptes.

Chaque enregistrement contient :

- l'URL ;
- le SHA-256 de l'archive ;
- le SHA-256 du binaire extrait ;
- la version et la plateforme.

Un binaire en cache n'est réutilisé que si son hash correspond. Les anciens pins.json contenant une simple chaîne de hash sont lus pour la migration, mais le nouveau format est écrit dans pins/.

TOFU

Sans pin connu, l'installation échoue par défaut. `trust_on_first_use = true` fait confiance au premier téléchargement HTTPS puis mémorise ses hashes.

Ce mode détecte une modification ultérieure. Il ne prouve pas que le premier artefact était légitime. Pour une chaîne de confiance plus forte, fournis `expected_sha256` depuis un canal indépendant.

Pour le préflight de release, `RUN_INSTALL_SMOKE=1 ./run_all_tests.sh` crée un cache temporaire vierge et exerce réellement le téléchargement, l'extraction, le `chmod`, le hash, la publication atomique et la relecture du pin pour geckodriver et chromedriver, sans toucher au cache utilisateur.

En CI, `BABET_WEBDRIVER_GITHUB_TOKEN`, `GH_TOKEN` ou `GITHUB_TOKEN` peut être défini pour authentifier la requête vers l'API GitHub utilisée pour geckodriver. Si un `GITHUB_TOKEN` GitHub Actions limité au dépôt courant est refusé par le dépôt public de geckodriver, la requête est retentée anonymement. Le token n'est jamais envoyé aux autres hôtes.

Extraction

L'archive n'est jamais entièrement chargée dans Lua. Babet la télécharge dans un temporaire, vérifie le hash, puis extrait uniquement vers un fichier temporaire unique avant publication atomique dans le cache :

- geckodriver pour Firefox ;
- chromedriver-<plateforme>/chromedriver pour Chrome.

Les limites anti-bombe sont appliquées à l'archive entière.

WebDriver BiDi

La version 2.0.0 ajoute un transport WebDriver BiDi sans remplacer WebDriver Classic. La session HTTP est créée normalement avec la capability `W3C webSocketUrl = true`, puis le client se connecte à l'URL renvoyée par le driver. Les commandes Classic et BiDi pilotent donc **le même navigateur et la même session**.

Négociation et client direct

```
local webdriver = require("webdriver")

local driver = assert(webdriver.firefox({
    headless = true,
    bidi = true,
}))
```

```
print(assert(driver:websocket_url()))
local bidi = assert(driver:bidi())
print(assert(bidi:status()).ready)
```

bidi = true nécessite Babet 2.22.0 ou supérieur et `babet.websocket`. Si le driver accepte la création de session mais ne renvoie pas de `WebSocketUrl`, la création échoue et `babet-webdriver` nettoie la session, le processus driver et le profil temporaire.

Options de connexion de `driver:bidi(options)` :

Option	Défaut	Description
<code>timeout</code>	5 s	timeout de connexion WebSocket
<code>command_timeout</code>	30 s	budget d'une commande BiDi
<code>close_timeout</code>	5 s	budget du handshake de fermeture
<code>event_queue_limit</code>	1024	nombre maximal d'événements mis en attente
<code>verify</code>	Babet	vérification TLS pour wss://
<code>ca_cert, ca_path, hostname, min_version</code>	Babet	paramètres TLS transmis au transport
<code>max_message_bytes, max_frame_bytes</code>	Babet	limites de taille du transport

Commandes typées

Le socle 2.0.0 fournit :

```
local status = assert(bidi:status())
local tree = assert(bidi:get_tree({ max_depth = 0 }))
local context = tree.contexts[1].context

local navigation = assert(bidi:navigate(
    context,
    "https://example.com",
    { wait = "complete" }
))

local evaluation = assert(bidi:evaluate("document.title", context))
local realms = assert(bidi:get_realms({ context = context }))
```

`evaluate()` accepte un identifiant de browsing context ou une table `target`. Les options prises en charge sont `await_promise`, `result_ownership`, `serialization_options` et `user_activation`.

Le standard BiDi évolue indépendamment des versions de `babet-webdriver`. Pour une commande encore non encapsulée, `call()` fournit l'accès bas niveau :

```
local result = assert(bidi:call("browser.getUserContexts", {}))
```

Souscriptions et événements

```
local subscription = assert(bidi:subscribe({
    "log.entryAdded",
    "network.beforeRequestSent",
}, {
    contexts = { context },
}))

local event, err, code = bidi:next_event(5)
if event then
    print(event.method)
elseif code ~= "timeout" then
    error(err)
```

```
end
```

```
assert(bidi:unsubscribe(subscription))
```

Les événements qui arrivent pendant l'attente d'une réponse sont conservés et ne désynchronisent pas les identifiants de commandes. La file est bornée par `event_queue_limit`. Si elle déborde, les événements excédentaires sont comptés et un événement synthétique `babetWebDriver.eventOverflow` signale la perte.

Une interface de callbacks explicite est également disponible :

```
bidi:on("log.entryAdded", function(event)
    print(event.params.text)
end)
```

```
assert(bidi:dispatch(5, 10))
```

`dispatch()` exécute les callbacks dans le thread Lua appelant. Il n'y a donc pas de callback implicite ou concurrent qui puisse interrompre arbitrairement une commande utilisateur.

Worker BiDi dédié

`driver:bidi_worker()` place la connexion WebSocket dans un worker distinct :

```
local driver = assert(webdriver.chromium({ headless = true, bidi = true }))
local bidi = assert(driver:bidi_worker({ event_queue_limit = 1024 }))
local subscription = assert(bidi:subscribe("log.entryAdded"))

local context = assert(bidi:get_tree({ max_depth = 0 })).contexts[1].context
assert(bidi:evaluate("console.log('hello bidi')", context))
local event = assert(bidi:next_event(5))
print(event.method)

assert(bidi:unsubscribe(subscription))
assert(driver:quit())
```

Avec une session Classic déjà hébergée par `webdriver_worker.lua`, le parent appelle simplement `session:bidi()`. Le worker BiDi est séparé du worker Classic et possède son propre WebSocket.

`babet.websocket` étant synchrone, ce worker est piloté par commandes : il attend son channel parent et ne lit le WebSocket que pendant une commande BiDi ou un appel à `next_event()`. Un `next_event(timeout)` long est lu par tranches bornées afin qu'une annulation du worker reste réactive. Il ne sonde jamais le réseau au repos, ce qui évite qu'un `recv()` WebSocket bloque la réception de la commande suivante. Les événements intercalés pendant une commande sont conservés dans la file bornée du client BiDi (`event_queue_limit`).

Les options propres au worker sont `channel_capacity`, `command_timeout`, `worker_start_timeout` et `stop_timeout`. Les options du client BiDi, notamment `event_queue_limit`, restent également acceptées et sont transmises au transport.

`driver:quit()` et `session:stop()` ferment automatiquement le transport BiDi attaché avant de terminer la session Classic. Les objets BiDi sont également compatibles avec la fermeture Lua 5.4/5.5 via `__close`.

Pour `webdriver_worker`, `stop_timeout` (ou le `timeout` explicite de `session:stop(timeout)`) est un budget global : il couvre la fermeture éventuelle du worker BiDi attaché, puis l'arrêt et la jointure du worker Classic.

Workers et channels

`webdriver_worker.lua` garde la session et le processus driver dans un worker persistant.

```

local worker_driver = require("webdriver_worker")
local session = assert(worker_driver.start({
    browser = "firefox",
    headless = true,
    channel_capacity = 64,
    command_timeout = 120,
}))

assert(session:open("https://example.com"))
local heading = assert(session:css("h1"))
print(assert(heading:text()))
assert(session:stop())

```

Le budget d'attente du parent couvre au minimum `request_timeout` avec une marge de transport. Pour `wait()`, il couvre aussi le timeout logique demandé et une dernière requête HTTP, afin que le proxy parent n'expire pas avant le worker.

Avant de créer le worker, le parent prépare le driver si nécessaire. Les installations concurrentes restent sûres : chaque extraction utilise son propre fichier temporaire et le binaire final est publié atomiquement.

Proxies d'éléments et de Shadow DOM

Un élément WebDriver n'est pas envoyé comme userdata. Le worker transmet son identifiant W3C et le parent crée un proxy :

```

local element = assert(session:css("h1"))
assert(worker_driver.is_element(element))
print(element:element_id())

```

Un ShadowRoot suit le même principe :

```

local shadow = assert(element:shadow_root())
assert(worker_driver.is_shadow_root(shadow))
local button = assert(shadow:find("button"))

```

Les proxies peuvent être réutilisés comme arguments de `js()` ou `frame()`. Le transport interne préserve aussi les arguments `nil`, y compris au milieu d'une liste, ainsi que les multiples valeurs de retour et le code d'erreur W3C.

Cycle de vie

```

print(session:status())
assert(session:stop(10))

```

`stop()` demande d'abord au worker de fermer proprement la session. En cas de timeout, l'annulation Babel est déclenchée. Elle reste coopérative : toutes les opérations HTTP du module sont donc bornées.

La session proxy est synchrone et ne doit avoir qu'une commande en vol. Pour le parallélisme, crée plusieurs sessions.

Le builder `actions()` n'est pas transporté par le proxy worker. Les actions chaînables restent disponibles sur une session directe.

Contrat d'erreur

Une erreur opérationnelle renvoie :

```

nil, "webdriver: ..."

```

Une réponse WebDriver conserve son code W3C dans le message et, pour les méthodes qui l'exposent comme `find()`, dans une troisième valeur de retour :


```
local element, err, code = driver:find("#absent")
assert(element == nil and code == "no such element")
```

Exemples de messages :

```
webdriver: no such element: élément absent
webdriver: stale element reference: ...
webdriver: invalid session id: ...
```

Erreurs de programmation levées :

- option inconnue ;
- mauvais type ;
- tableau d'arguments troué ;
- stratégie de recherche inconnue ;
- frame d'un type invalide ;
- table cyclique envoyée à JavaScript.

Limites

- le client BiDi direct est synchrone ; utiliser `bidi_worker()` pour isoler la boucle WebSocket ;
- la surface typée BiDi 2.0.0 couvre le socle `session/browsingContext/script;call()` expose le reste du protocole ;
- Linux seulement dans le gestionnaire automatique actuel ;
- Chrome for Testing ne fournit qu'un binaire Linux x86-64 ;
- pas de proxy distant authentifié ou TLS pour le serveur WebDriver ;
- pas de reprise de téléchargement ;
- pas de callback de progression ;
- proxy worker limité aux méthodes sérialisables et sans builder Actions ;
- les channels Babet transportent du JSON-like, pas des octets binaires avec NUL.

Session WebDriver dans un worker

Le parcours d'intégration complet peut être vérifié avec :

```
HEADLESS=1 ./run_worker_smoke.sh firefox
HEADLESS=1 ./run_worker_smoke.sh chromium
```

Ce test couvre le parent, les channels, le worker, le processus driver et le navigateur réel.